SQL (Basic)

Objective

By the end of this training, trainees will be able to write SQL queries confidently - from simple SELECT statements to advanced joins, subqueries, window functions, and CTEs - and will understand core database concepts (tables, keys, constraints, transactions, and indexing) well enough to work with real production databases.

Outline

1. Introduction to SQL
   - 1.1. What is a Database
   - 1.2. Types of SQL Commands (DDL, DML, DCL, TCL)
   - 1.3. How to Follow Along
2. Getting Started
   - 2.1. Sample Database
3. Database Concepts
   - 3.1. Common Data Types
4. SELECT Statement
   - 4.1. Selecting All Columns (*)
   - 4.2. Column Aliasing (AS)
5. Filtering Data
   - 5.1. WHERE Clause
   - 5.2. Comparison Operators
   - 5.3. Logical Operators (AND, OR, NOT)
   - 5.4. BETWEEN, IN, LIKE
   - 5.5. IS NULL / IS NOT NULL
6. Sorting and Limiting Results
   - 6.1. ORDER BY
   - 6.2. LIMIT
   - 6.3. DISTINCT
7. Aggregate Functions
8. Grouping Data
   - 8.1. GROUP BY
   - 8.2. HAVING
9. Joins
   - 9.1. INNER JOIN
   - 9.2. LEFT JOIN
   - 9.3. RIGHT JOIN
10. Modifying Data
    - 10.1. INSERT
    - 10.2. UPDATE
    - 10.3. DELETE
11. Creating Tables and Constraints
12. SQL Clause Execution Order
    - 12.1. Order You Write vs. Order SQL Executes

12.2. Why Clause Order Matters
13. Summary
14. Gem
15. Reference Material
16. Advanced Learning Material

1. Introduction

SQL (Structured Query Language) is the standard language used to talk to databases. Think of a database as a giant, extremely well-organized set of spreadsheets (called tables) that store information - like employee records, orders, products, or student grades. SQL is how we ask questions of that data (for example, "show me all employees earning more than $80,000") and how we add, change, or remove data. This guide uses one simple, consistent sample database (an Employees and Departments table) throughout, so every concept builds naturally on the last - no prior SQL experience required. SQL is used everywhere - web applications, mobile applications, analytics dashboards, and data science all rely on it. It works across most major database systems (MySQL, PostgreSQL, SQL Server, Oracle, and Trino/Presto), enabling you to answer business questions directly from data without writing a full application.

What is a Database?

A database is an organized collection of data stored electronically. A Database Management System (DBMS) - such as MySQL, PostgreSQL, or SQL Server - is the software that lets you create, read, update, and delete that data safely and efficiently. Inside a database, data is organized into tables, similar to sheets in an Excel workbook. Each table has columns (fields) and rows (records).

Types of SQL Commands

Category	Purpose	Example Commands
DDL - Data Definition Language	Define or change the structure of the database	CREATE, ALTER, DROP
DML - Data Manipulation Language	Work with the data itself	SELECT, INSERT, UPDATE, DELETE
DCL - Data Control Language	Control access and permissions	GRANT, REVOKE
TCL - Transaction Control Language	Manage groups of changes as a unit	COMMIT, ROLLBACK

Note: *This guide focuses mainly on DML (especially SELECT, which you will use most often) plus enough DDL and TCL to be a well-rounded SQL user.*

How to Follow Along

Every example in this guide uses one simple, consistent sample database (introduced next) so you can see how each concept builds on the last.

MySQL Setup :

For the best learning experience, install MySQL Community Server and MySQL Workbench on your local machine before starting the hands-on exercises.

- Download the MySQL Installer from the official MySQL website.
- Install MySQL Community Server and MySQL Workbench using the default installation options.
- During installation, set a password for the root user and remember it for future use.
- Open MySQL Workbench and create a connection to your local MySQL server using the root credentials.
- Create a new database (for example, sql_training).
- Run the sample SQL scripts provided in this guide to create the required tables and insert the sample data.
- Execute the SQL queries from each section and compare your results with the expected outputs.

Note: *If you prefer not to install MySQL, you can practice using online SQL editors such as DB Fiddle, SQLite Online, or the W3Schools Try SQL Editor.*

2. Getting Started: Our Sample Database

Throughout this guide we use a small, easy-to-understand HR database with two tables: Departments and Employees. Every employee belongs to a department, and every employee (except the top manager) reports to another employee (their manager). This simple design is enough to demonstrate every SQL concept from beginner to advanced.

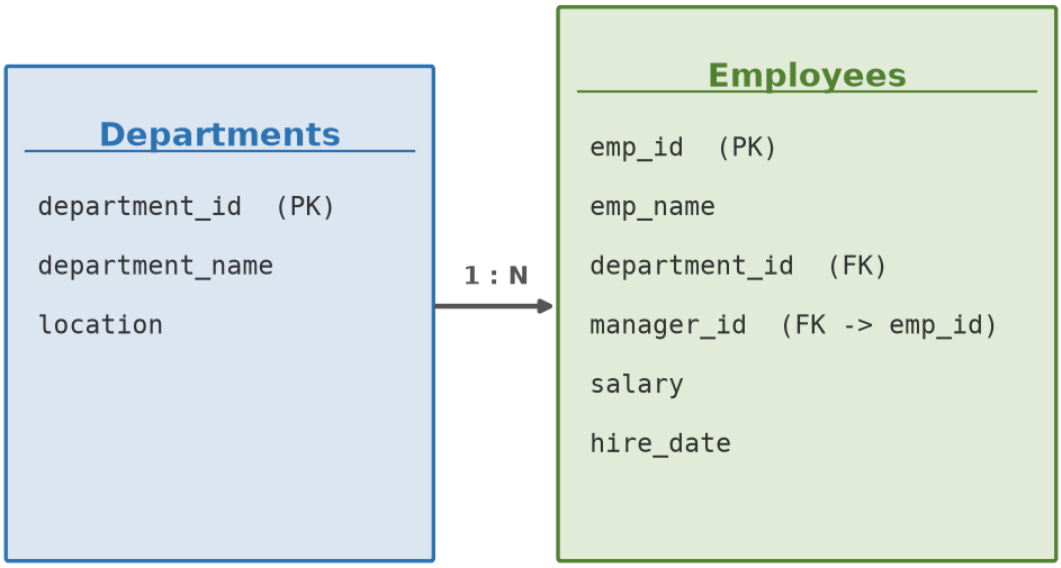


Figure 1: Entity Relationship (ER) diagram of our sample database. One department can have many employees (a 1:N relationship).

Departments Table

department_id	department_name	location

1	Engineering	Bangalore
2	Sales	Mumbai
3	HR	Delhi
4	Marketing	Pune

Employees Table

emp_id	emp_name	department_id	manager_id	salary	hire_date
101	Priya Sharma	1	NULL	150000	2018-01-15
102	Amit Verma	1	101	90000	2019-03-22
103	Neha Gupta	1	101	95000	2020-07-01
104	Rahul Singh	2	101	60000	2019-11-10
105	Sara Khan	2	104	65000	2021-02-18
106	Vikram Rao	3	101	70000	2017-06-05
107	Anjali Nair	4	101	72000	2022-01-30

Tip: *We reuse these exact two tables in every chapter. Try to predict the output of each query yourself before checking the sample result - this is the fastest way to learn SQL.*

3. Database Concepts: Tables, Rows, Columns, and Keys

Before writing queries, it helps to know the vocabulary used to describe a table's structure.
Table - a collection of related data organized in rows and columns, such as our Employees table.
Row (record) - a single entry in a table - one employee, one order, one product.
Column (field) - a single piece of information stored for every row, such as emp_name or salary. Every column has a data type (see below).
Primary Key (PK) - a column (or set of columns) that uniquely identifies each row in a table. In our Employees table, emp_id is the primary key - no two employees share the same emp_id.
Foreign Key (FK) - a column in one table that refers to the primary key of another table, creating a link between the two. department_id in Employees is a foreign key that points to department_id in Departments.

Common Data Types

Data Type	Used For	Example
INT / INTEGER	Whole numbers	101, 90000
VARCHAR(n) / TEXT	Text of variable length	'Priya Sharma'

DECIMAL / NUMERIC	Precise decimal numbers (money)	89999.50
DATE	Calendar dates	'2020-07-01'
BOOLEAN	True/false values	TRUE, FALSE

4. The SELECT Statement

SELECT is the command you will use most in SQL - it retrieves data from one or more tables. At its simplest, you tell SQL which columns you want and which table to get them from.

```
SELECT emp_name, salary
FROM Employees;
```

emp_name	salary
Priya Sharma	150000
Amit Verma	90000
Neha Gupta	95000
Rahul Singh	60000
Sara Khan	65000
Vikram Rao	70000
Anjali Nair	72000

Selecting All Columns

Use the * wildcard to return every column without typing each name:

```
SELECT * FROM Employees;
```

Aliasing Columns

Use **AS** to give a column (or table) a friendlier name in the output. This does not change the data, only how it is displayed.

```
SELECT emp_name AS employee, salary AS annual_salary
FROM Employees;
```

Note: *AS is optional in most databases - SELECT emp_name employee works too - but writing AS explicitly makes queries easier to read.*

5. Filtering Data

Where Clause

The WHERE clause lets you filter rows so that only the ones matching a condition are returned.

```
SELECT emp_name, salary
FROM Employees
WHERE salary > 80000;
```

emp_name	salary
Priya Sharma	150000
Amit Verma	90000
Neha Gupta	95000

Comparison Operators

Operator	Meaning	Example
=	Equal to	department_id = 1
<> or !=	Not equal to	department_id <> 1
>, <	Greater / less than	salary > 70000
>=, <=	Greater / less than or equal to	salary >= 70000

Combining Conditions: AND, OR, NOT

Use AND when every condition must be true, OR when at least one must be true, and NOT to reverse a condition.

```
SELECT emp_name, department_id, salary
FROM Employees
WHERE department_id = 1 AND salary > 90000;
```

BETWEEN, IN, and LIKE

BETWEEN - checks whether a value falls within a range (inclusive).

```
SELECT emp_name, salary
FROM Employees
WHERE salary BETWEEN 60000 AND 90000;
```

IN - checks whether a value matches any item in a list - a shorthand for multiple OR conditions.

```
SELECT emp_name, department_id
```

```
FROM Employees
WHERE department_id IN (1, 3);
```

LIKE - matches text patterns using wildcards: % means any number of characters, _ means exactly one character.

```
SELECT emp_name
FROM Employees
WHERE emp_name LIKE 'A%';   – names starting with 'A'
```

emp_name
Amit Verma
Anjali Nair

NULL checks - NULL means "no value" and must be tested with IS NULL or IS NOT NULL - never with = NULL.

```
SELECT emp_name
FROM Employees
WHERE manager_id IS NULL;   – finds the top-level manager
```

6. Sorting and Limiting Results

ORDER BY

ORDER BY sorts the result set by one or more columns. Add DESC for descending order; ascending (ASC) is the default.

```
SELECT emp_name, salary
FROM Employees
ORDER BY salary DESC;
```

emp_name	salary
Priya Sharma	150000
Neha Gupta	95000
Amit Verma	90000
Anjali Nair	72000
Vikram Rao	70000
Sara Khan	65000
Rahul Singh	60000

LIMIT (Top N Rows)

LIMIT restricts how many rows are returned - handy for "top 3" style questions. (SQL Server uses TOP 3 instead of LIMIT 3.)

```
SELECT emp_name, salary
FROM Employees
ORDER BY salary DESC
LIMIT 3;
```

DISTINCT

DISTINCT removes duplicate rows from the result, keeping only unique values.

```
SELECT DISTINCT department_id
FROM Employees;
```

department_id
1
2
3
4

7. Aggregate Functions

Aggregate functions calculate a single summary value from many rows - useful for totals, averages, counts, and finding extremes.

Function	Purpose
COUNT()	Number of rows
SUM()	Total of a numeric column
AVG()	Average of a numeric column
MIN()	Smallest value
MAX()	Largest value

```
SELECT
   COUNT(*)      AS total_employees,
   SUM(salary)   AS total_payroll,
   AVG(salary)   AS avg_salary,
   MIN(salary)   AS lowest_salary,
   MAX(salary)   AS highest_salary
FROM Employees;
```

total_employees	total_payroll	avg_salary	lowest_salary	highest_salary
7	602000	86000	60000	150000

Note: *Aggregate functions ignore NULL values by default (except COUNT(*), which counts every row regardless of NULLs).*

8. Grouping Data

GROUP BY

GROUP BY splits rows into buckets that share the same value in a column, then applies an aggregate function to each bucket separately - for example, the average salary per department.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM Employees
GROUP BY department_id;
```

department_id	avg_salary
1	111666.67
2	62500
3	70000
4	72000

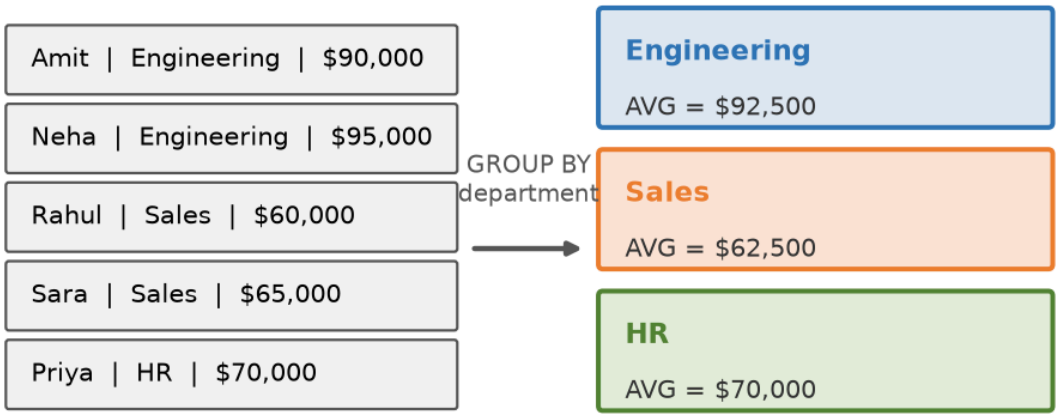


Figure 2: GROUP BY collects rows that share the same department into buckets, then computes one aggregate value per bucket.

Filtering Groups with HAVING

WHERE filters individual rows before grouping; HAVING filters groups after aggregation. Use HAVING when your condition involves an aggregate function.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM Employees
GROUP BY department_id
```

```
HAVING AVG(salary) > 65000;
```

department_id	avg_salary
1	111666.67
3	70000
4	72000

Note: *WHERE cannot use aggregate functions like AVG(salary) - that is exactly what HAVING is for.*

9. Basic Joins: INNER, LEFT, and RIGHT

A join combines rows from two or more tables based on a related column - in our case, department_id links Employees to Departments. Joins are one of the most powerful (and most tested) SQL concepts, so we introduce them here and go further in Part 2.

INNER JOIN

Returns only the rows that have a match in both tables.

```
SELECT e.emp_name, d.department_name
FROM Employees e
INNER JOIN Departments d
   ON e.department_id = d.department_id;
```

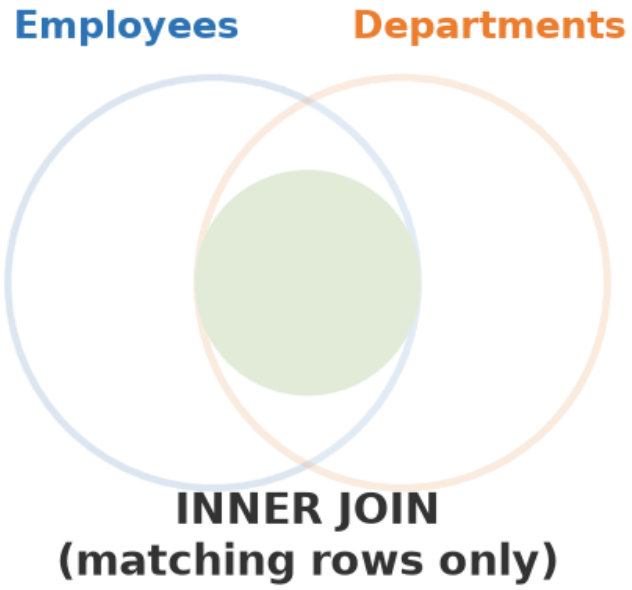


Figure 3: INNER JOIN keeps only the rows where Employees and Departments match.

emp_name	department_name
Priya Sharma	Engineering
Amit Verma	Engineering
Neha Gupta	Engineering
Rahul Singh	Sales
Sara Khan	Sales
Vikram Rao	HR
Anjali Nair	Marketing

LEFT JOIN

Returns every row from the left table (Employees), plus matching data from the right table (Departments). If there is no match, the right side's columns show NULL.

```
SELECT e.emp_name, d.department_name
FROM Employees e
LEFT JOIN Departments d
   ON e.department_id = d.department_id;
```

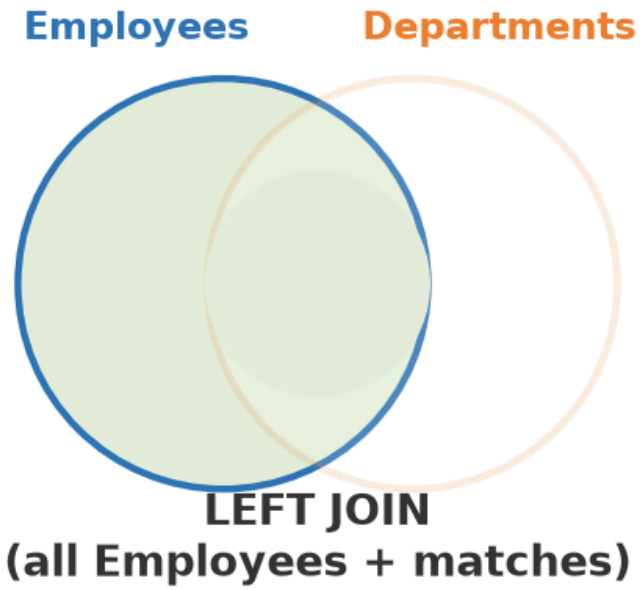


Figure 4: LEFT JOIN keeps all Employees rows, matching Departments data where it exists.

RIGHT JOIN

The mirror image of LEFT JOIN: returns every row from the right table (Departments), plus matching data from the left table (Employees). Any department with zero employees would still appear, with NULLs for the employee columns.

```
SELECT e.emp_name, d.department_name
FROM Employees e
RIGHT JOIN Departments d
   ON e.department_id = d.department_id;
```

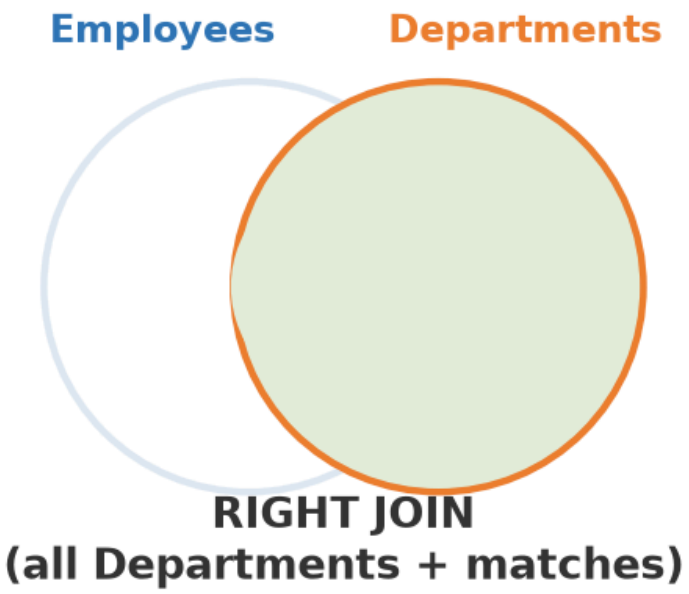


Figure 5: RIGHT JOIN keeps all Departments rows, matching Employees data where it exists.

Note: *Not all databases support RIGHT JOIN (e.g., older SQLite versions) - you can usually rewrite a RIGHT JOIN as a LEFT JOIN by swapping the table order.*

10. Modifying Data: INSERT, UPDATE, DELETE

INSERT - Adding New Rows

```
INSERT INTO Employees (emp_id, emp_name, department_id, manager_id, salary, hire_date)
VALUES (108, 'Karan Mehta', 2, 104, 58000, '2023-05-12');
```

UPDATE - Changing Existing Rows

Always include a WHERE clause with UPDATE - without one, every row in the table gets updated!

```
UPDATE Employees
SET salary = 63000
WHERE emp_id = 108;
```

DELETE - Removing Rows

```
DELETE FROM Employees
WHERE emp_id = 108;
```

Warning: *DELETE without a WHERE clause removes every row in the table. Always double-check your WHERE clause (or run a SELECT with the same condition first) before running UPDATE or DELETE.*

11. Creating Tables and Constraints

CREATE TABLE defines a brand-new table, including each column's name, data type, and any rules (constraints) that keep the data valid.

```
CREATE TABLE Departments (
    department_id   INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL,
    location        VARCHAR(50)
);

CREATE TABLE Employees (
    emp_id          INT PRIMARY KEY,
    emp_name        VARCHAR(100) NOT NULL,
    department_id   INT,
    manager_id      INT,
    salary          DECIMAL(10,2) DEFAULT 0,
    hire_date       DATE,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id),
    FOREIGN KEY (manager_id) REFERENCES Employees(emp_id)
);
```

Common Constraints

Constraint	Meaning
PRIMARY KEY	Uniquely identifies each row; cannot be NULL or duplicated
FOREIGN KEY	Links this column to a primary key in another table
NOT NULL	The column must always have a value
UNIQUE	No two rows may share the same value in this column
DEFAULT	A fallback value used when none is provided on INSERT

12. Clause Priority: How SQL Actually Executes a Query

Even though we type a SELECT statement with its clauses in a fixed written order (SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT), the database does not run them in that order. Understanding the real order of execution explains several common beginner questions - for example, why a WHERE clause cannot use an aggregate function, or why it cannot reference a column alias defined in SELECT.

Order You Write vs. Order SQL Executes

Step	Clause	What It Does
1	FROM	Identifies the source table(s) to read from
2	JOIN	Combines rows from multiple tables
3	WHERE	Filters individual rows before any grouping happens
4	GROUP BY	Groups the remaining rows into buckets
5	HAVING	Filters groups after aggregation
6	SELECT	Chooses and computes the final output columns
7	DISTINCT	Removes duplicate rows from the result
8	ORDER BY	Sorts the final result set
9	LIMIT / OFFSET	Restricts how many rows are returned

Why This Matters

WHERE runs before SELECT, so it cannot reference a column alias you defined in SELECT - that alias does not exist yet.
WHERE also runs before GROUP BY, so it cannot use aggregate functions like AVG() or SUM() - use HAVING for that instead (see the earlier GROUP BY and HAVING section).
ORDER BY runs last (almost), so it can freely use column aliases defined in SELECT.
LIMIT is applied only after everything else - including sorting - so it always returns the correct "top N" rows.
Example: this single query touches almost every clause. The comments show the real execution order, not the order the lines are written in.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM Employees                -- 1. FROM: start with Employees
WHERE salary > 50000            -- 3. WHERE: filter rows first
```

```
GROUP BY department_id          —- 4. GROUP BY: bucket by department
HAVING AVG(salary) > 65000      —- 5. HAVING: filter the buckets
ORDER BY avg_salary DESC;       —- 8. ORDER BY: sort the final output
```

13. Summary

In this module, we learned the fundamentals of SQL, including database concepts, data retrieval using SELECT, filtering, sorting, aggregate functions, grouping, joins, data manipulation (INSERT, UPDATE, DELETE), table creation with constraints, and the SQL query execution order. These concepts provide a strong foundation for writing and understanding SQL queries.

14. Gem

- https://gemini.google.com/gem/1spEHko-qGnCsm-yGC7LwSFDeeHRuScEk?usp=sharing

15. Reference Material

- W3Schools SQL Tutorial - https://www.w3schools.com/sql/
- GeeksforGeeks SQL Tutorial - https://www.geeksforgeeks.org/sql/sql-tutorial/

16. Advanced Learning Material

- code academy - https://www.codecademy.com/learn/learn-sql